Moving Code to a Sidekiq Job

A common use of Sidekiq jobs is to move long-running code out of the web request/response cycle and into a background job to free up web server resources to handle other requests. This often means that the user can get a response much more quickly since they won't have to wait for the slow code from, say, the payments service, to complete. Let's move all three HTTP calls out of the main request/response path and into a Sidekiq job.

Create a New Sidekiq Job

The example app's order creation logic makes calls to payments, email, and order fulfillment services over HTTP. If these were real services provided by third parties, the order creation feature would be very slow, and our app's web servers would spend a lot of time doing nothing (meaning you'd need a lot of web servers to sit around doing nothing to handle the app's traffic).

Here's the main logic of how everything works currently:

snapshots/1-2/sidekiq-book/app/services/order_creator.rb

```
def create_order(order)
  if order.save
    payments_response = charge(order)
    if payments_response.success?

      email_response       = send_email(order)
      fulfillment_response = request_fulfillment(order)

      order.update!(
        charge_id: payments_response.charge_id,
        charge_completed_at: Time.zone.now,
        charge_successful: true,
        email_id: email_response.email_id,
        fulfillment_request_id: fulfillment_response.request_id)
    else
      order.update!(
        charge_completed_at: Time.zone.now,
        charge_successful: false,
```